

Signals and Their Properties

Beginner Problem Set (10 Problems)

Reference solutions, output, and figures

Warm-up problems to master the basics before the advanced set

Complete solutions to the beginner set. Each program was executed and its self-checks verified; the console output and figure shown are the actual results. Compare against your own attempt — the ideas here are exactly the ones scaled up in the advanced problem set.

Problem 01 — Generating and Plotting Basic Signals

Problem 01

(Marks: 10 Time: 15 min)

Every signals course begins by creating a time axis and evaluating a few standard signals. Here you build a sine, a cosine, and an exponential, then plot them on one figure.

Tasks.

1. Implement `make_sine` and `make_cosine` for $A \sin(2\pi ft + \phi)$ and $A \cos(2\pi ft + \phi)$.
2. Implement `make_exp` for Ae^{at} (decaying when $a < 0$).
3. Implement `plot_signals` to draw a dictionary of `{label: signal}` with labels, legend, and grid.

Marks Distribution: sine 2; cosine 2; exponential 3; plotting 3.

Solution

A sine, a cosine, and a decaying exponential are the three signals you will meet again and again. Note $\sin(0) = 0$, $\cos(0) = 1$, and $e^0 = 1$: quick sanity checks that your functions are wired correctly.

```

1  """P01 - Generating and plotting basic signals. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6
7  # =====
8  # ANSWER IMPLEMENTATION
9  # =====
10 def make_sine(t, A, f, phi):
11     """A sine wave:  $A * \sin(2\pi f * t + \phi)$ ."""
12     return A * np.sin(2 * np.pi * f * t + phi)
13
14
15 def make_cosine(t, A, f, phi):
16     """A cosine wave:  $A * \cos(2\pi f * t + \phi)$ ."""
17     return A * np.cos(2 * np.pi * f * t + phi)
18
19
20 def make_exp(t, A, a):
21     """An exponential:  $A * \exp(a * t)$  (decaying when  $a < 0$ )."""
22     return A * np.exp(a * t)
23
24
25 def plot_signals(t, signals, title):
26     """signals is a dict {label: array}. Plot each on one figure."""
27     fig, ax = plt.subplots(figsize=(9, 4))

```

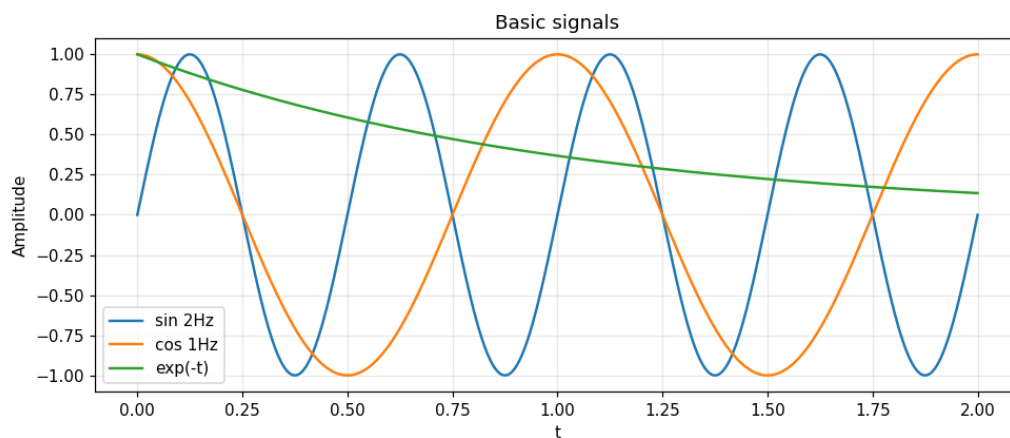
```

28     for label, y in signals.items():
29         ax.plot(t, y, label=label, lw=1.6)
30     ax.set_title(title); ax.set_xlabel("t"); ax.set_ylabel("Amplitude")
31     ax.grid(True, alpha=0.3); ax.legend()
32     plt.show()
33
34
35 def main():
36     t = np.linspace(0, 2, 1000)
37     s = make_sine(t, A=1.0, f=2.0, phi=0.0)
38     c = make_cosine(t, A=1.0, f=1.0, phi=0.0)
39     e = make_exp(t, A=1.0, a=-1.0)
40     plot_signals(t, {"sin 2Hz": s, "cos 1Hz": c, "exp(-t)": e},
41                   "Basic signals")
42
43     # self-checks
44     assert np.isclose(make_sine(np.array([0.0]), 1, 2, 0)[0], 0.0)
45     assert np.isclose(make_cosine(np.array([0.0]), 1, 1, 0)[0], 1.0)
46     assert np.isclose(make_exp(np.array([0.0]), 3, -1)[0], 3.0)
47     print("[P01] sin(0)=0, cos(0)=1, exp(0)*3=3 -> all checks passed.")
48
49
50 if __name__ == "__main__":
51     main()

```

Console output:

```
[P01] sin(0)=0, cos(0)=1, exp(0)*3=3 -> all checks passed.
```



Problem 02 — Amplitude Scaling and DC Offset

Problem 02

(Marks: 10 Time: 15 min)

The two simplest operations on a signal: multiply it by a constant (amplitude scaling) and add a constant (a DC offset), giving $y(t) = Ax(t) + B$.

Tasks.

1. Implement `amplitude_scale(x, A) = Ax`.
2. Implement `add_offset(x, B) = x + B`.
3. Implement `transform(x, A, B) = Ax + B` by reusing the two above.
4. Plot x , $2x$, $x + 1$, and $2x + 1$ together.

Marks Distribution: scaling 3; offset 3; combined 2; plot 2.

Solution

Amplitude scaling stretches the signal vertically; a DC offset lifts the whole curve. Adding $B = 1$ to a sine (whose average is 0) raises its average to 1 – a good way to check your offset.

```

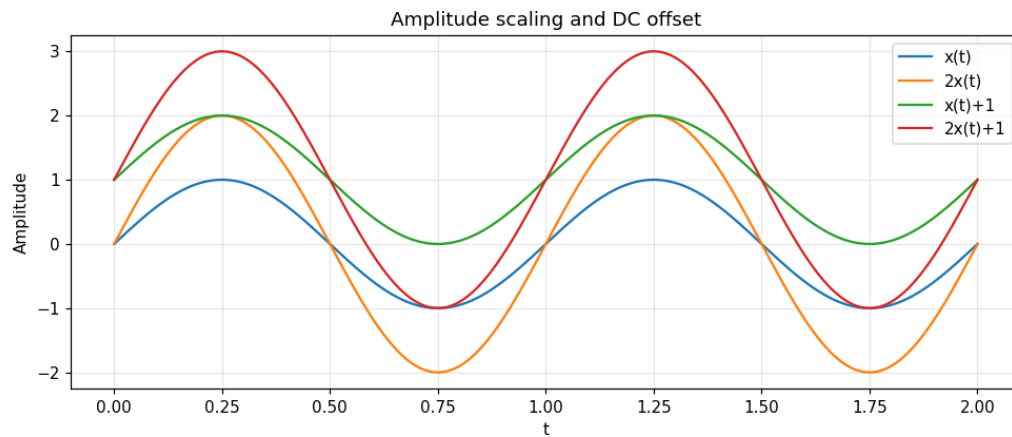
1  """P02 - Amplitude scaling and DC offset  y = A*x + B. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6  T_MIN, T_MAX, N = 0.0, 2.0, 1000
7
8
9  def x_of_t(t):
10     """Base signal (given): a 1 Hz sine."""
11     return np.sin(2 * np.pi * 1.0 * t)
12
13
14  # =====
15  # ANSWER IMPLEMENTATION
16  # =====
17  def amplitude_scale(x, A):
18     """Multiply every sample by A."""
19     return A * x
20
21
22  def add_offset(x, B):
23     """Add the constant B to every sample (a DC offset)."""
24     return x + B
25
26
27  def transform(x, A, B):
28     """Combined amplitude scaling and offset: A*x + B."""
29     return add_offset(amplitude_scale(x, A), B)
30
31
32  def main():
33     t = np.linspace(T_MIN, T_MAX, N)
34     x = x_of_t(t)
35
36     fig, ax = plt.subplots(figsize=(9, 4))
37     ax.plot(t, x, label="x(t)")
38     ax.plot(t, amplitude_scale(x, 2.0), label="2x(t)")
39     ax.plot(t, add_offset(x, 1.0), label="x(t)+1")
40     ax.plot(t, transform(x, 2.0, 1.0), label="2x(t)+1")
41     ax.set_title("Amplitude scaling and DC offset")
42     ax.set_xlabel("t"); ax.set_ylabel("Amplitude")
43     ax.grid(True, alpha=0.3); ax.legend()
44     plt.show()
45

```

```
46     assert np.allclose(transform(x, 2.0, 1.0), 2 * x + 1)
47     assert np.isclose(add_offset(x, 1.0).mean(), 1.0, atol=1e-2) # mean of sine ~0 +1
48     print("[P02] transform matches 2x+1; offset raises the mean to ~1 -> passed.")
49
50
51 if __name__ == "__main__":
52     main()
```

Console output:

```
[P02] transform matches 2x+1; offset raises the mean to ~1 -> passed.
```



Problem 03 — Building Piecewise Signals with Masks

Problem 03

(Marks: 10 Time: 20 min)

Many useful signals are defined piece by piece. Using boolean masks or `np.where`, build a rectangular pulse, a triangular pulse, and the unit step.

Tasks.

1. Implement `rect_pulse(t, width)`: 1 for $|t| \leq \text{width}/2$, else 0.
2. Implement `triangle_pulse(t, width)`: peaks at 1 at $t = 0$ and falls to 0 at $|t| = \text{width}/2$.
3. Implement `unit_step(t)`: 1 for $t \geq 0$, else 0.
4. Plot all three on one figure.

Marks Distribution: rect 3; triangle 4; step 1; plot 2.

Solution

The pattern `np.where(condition, value_if_true, value_if_false)` is the workhorse for piecewise signals. The triangle is just $1 - |t|/(\text{half width})$ clipped to zero outside the base.

```

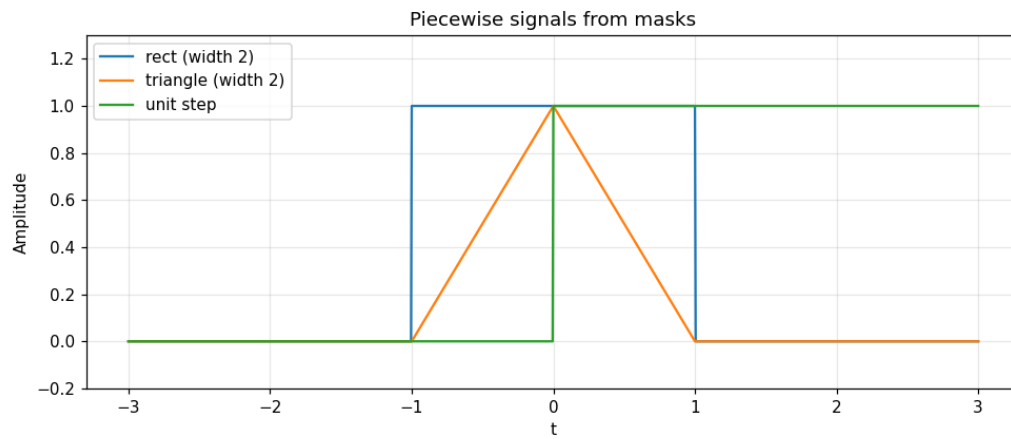
1  """P03 - Building piecewise signals with masks. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6  T_MIN, T_MAX, N = -3.0, 3.0, 1201
7
8
9  # =====
10 # ANSWER IMPLEMENTATION
11 # =====
12 def rect_pulse(t, width):
13     """1 where |t| <= width/2, else 0."""
14     return np.where(np.abs(t) <= width / 2, 1.0, 0.0)
15
16
17 def triangle_pulse(t, width):
18     """A triangle peaking at 1 for t=0, reaching 0 at |t| = width/2."""
19     half = width / 2
20     y = 1.0 - np.abs(t) / half
21     return np.where(np.abs(t) <= half, y, 0.0)
22
23
24 def unit_step(t):
25     """The Heaviside step: 1 for t >= 0, else 0."""
26     return np.where(t >= 0, 1.0, 0.0)
27
28
29 def main():
30     t = np.linspace(T_MIN, T_MAX, N)
31     fig, ax = plt.subplots(figsize=(9, 4))
32     ax.plot(t, rect_pulse(t, 2.0), label="rect (width 2)")
33     ax.plot(t, triangle_pulse(t, 2.0), label="triangle (width 2)")
34     ax.plot(t, unit_step(t), label="unit step")
35     ax.set_title("Piecewise signals from masks")
36     ax.set_xlabel("t"); ax.set_ylabel("Amplitude")
37     ax.set_ylim(-0.2, 1.3); ax.grid(True, alpha=0.3); ax.legend()
38     plt.show()
39
40     assert rect_pulse(np.array([0.0, 0.9, 1.1]), 2.0).tolist() == [1.0, 1.0, 0.0]
41     assert np.isclose(triangle_pulse(np.array([0.0]), 2.0)[0], 1.0)
42     assert np.isclose(triangle_pulse(np.array([0.5]), 2.0)[0], 0.5)
43     assert unit_step(np.array([-1.0, 0.0, 1.0])).tolist() == [0.0, 1.0, 1.0]
44     print("[P03] rect, triangle, and step take the expected values -> passed.")
45

```

```
46  
47 if __name__ == "__main__":  
48     main()
```

Console output:

```
[P03] rect, triangle, and step take the expected values -> passed.
```



Problem 04 — Adding and Multiplying Signals

Problem 04

(Marks: 10 Time: 15 min)

Signals combine pointwise: addition is superposition, multiplication is modulation. Add a slow and a fast cosine, then multiply them and watch the fast wave ride inside the slow envelope.

Tasks.

1. Implement `add_signals(a, b) = a + b`.
2. Implement `multiply_signals(a, b) = a · b`.
3. Plot the sum $x_1 + x_2$ on one axis and the product $x_1 x_2$ on another, overlaying $\pm x_1$ as the envelope.

Marks Distribution: add 2; multiply 3; plots 3; seeing the envelope 2.

Solution

Adding two cosines superposes them; multiplying them produces a fast oscillation whose amplitude is shaped by the slow signal – the essence of amplitude modulation. The product stays between $+x_1$ and $-x_1$.

```

1  """P04 - Adding and multiplying signals. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6  T_MIN, T_MAX, N = 0.0, 1.0, 2000
7
8
9  def x1_of_t(t): # given: slow cosine
10     return np.cos(2 * np.pi * 3 * t)
11
12
13  def x2_of_t(t): # given: fast cosine
14     return np.cos(2 * np.pi * 20 * t)
15
16
17  # =====
18  # ANSWER IMPLEMENTATION
19  # =====
20  def add_signals(a, b):
21     """Elementwise sum of two signals."""
22     return a + b
23
24
25  def multiply_signals(a, b):
26     """Elementwise product of two signals."""
27     return a * b
28
29
30  def main():
31     t = np.linspace(T_MIN, T_MAX, N)
32     x1, x2 = x1_of_t(t), x2_of_t(t)
33
34     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6))
35     ax1.plot(t, x1, label="x1 (3 Hz)")
36     ax1.plot(t, x2, label="x2 (20 Hz)", alpha=0.6)
37     ax1.plot(t, add_signals(x1, x2), label="x1 + x2", color="k", lw=1.2)
38     ax1.set_title("Sum of two signals"); ax1.legend(fontsize=8)
39     ax1.grid(True, alpha=0.3); ax1.set_xlabel("t")
40
41     ax2.plot(t, multiply_signals(x1, x2), color="C3")
42     ax2.plot(t, x1, "k--", alpha=0.5, label="x1 (envelope)")
43     ax2.plot(t, -x1, "k--", alpha=0.5)
44     ax2.set_title("Product of two signals (amplitude modulation)")

```

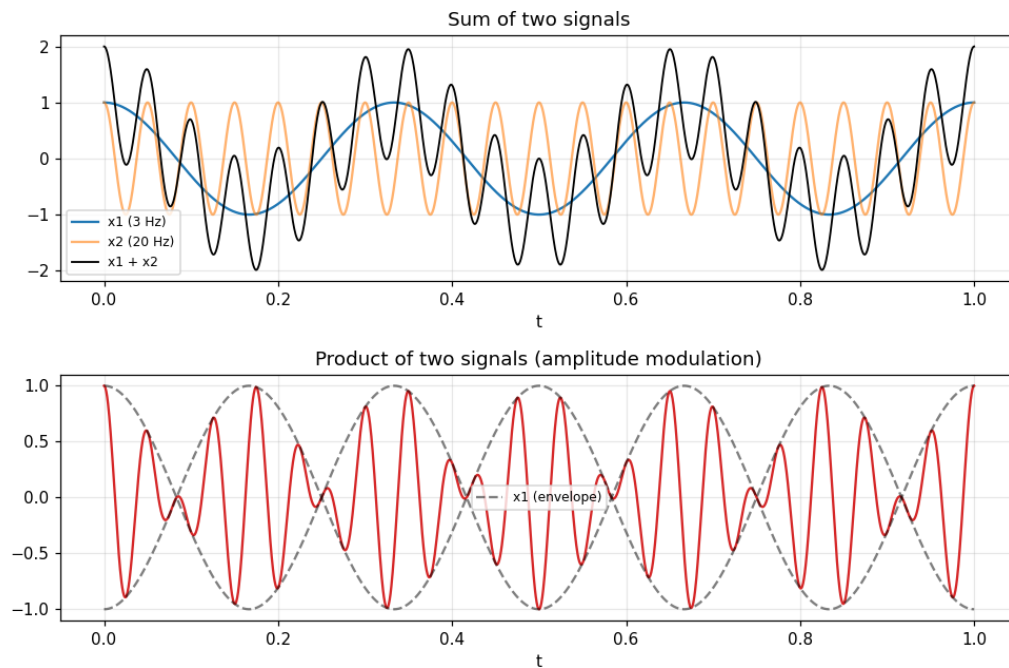
```

45 ax2.legend(fontsize=8); ax2.grid(True, alpha=0.3); ax2.set_xlabel("t")
46 plt.show()
47
48 assert np.allclose(add_signals(x1, x2), x1 + x2)
49 assert np.allclose(multiply_signals(x1, x2), x1 * x2)
50 print("[P04] sum and product computed; product rides inside the x1 envelope"
51       " -> passed.")
52
53
54 if __name__ == "__main__":
55     main()

```

Console output:

```
[P04] sum and product computed; product rides inside the x1 envelope -> passed.
```



Problem 05 — Discrete-Time Building Blocks

Problem 05

(Marks: 10 Time: 20 min)

The three atoms of discrete-time signals are the impulse $\delta[n]$, the step $u[n]$, and the rectangular pulse $u[n] - u[n - N]$. Build each and display it with a stem plot.

Tasks.

1. Implement `unit_impulse(n)`: 1 at $n = 0$, else 0.
2. Implement `unit_step(n)`: 1 for $n \geq 0$, else 0.
3. Implement `rect_pulse_dt(n, N)` as $u[n] - u[n - N]$.
4. Stem-plot all three (the `stem` helper is provided).

Marks Distribution: impulse 2; step 2; rect 3; stem plots 3.

Solution

The impulse has a single nonzero sample, so it sums to 1. Writing the rectangular pulse as a difference of two steps, $u[n] - u[n - N]$, is a habit worth forming: it generalises to any windowing operation.

```

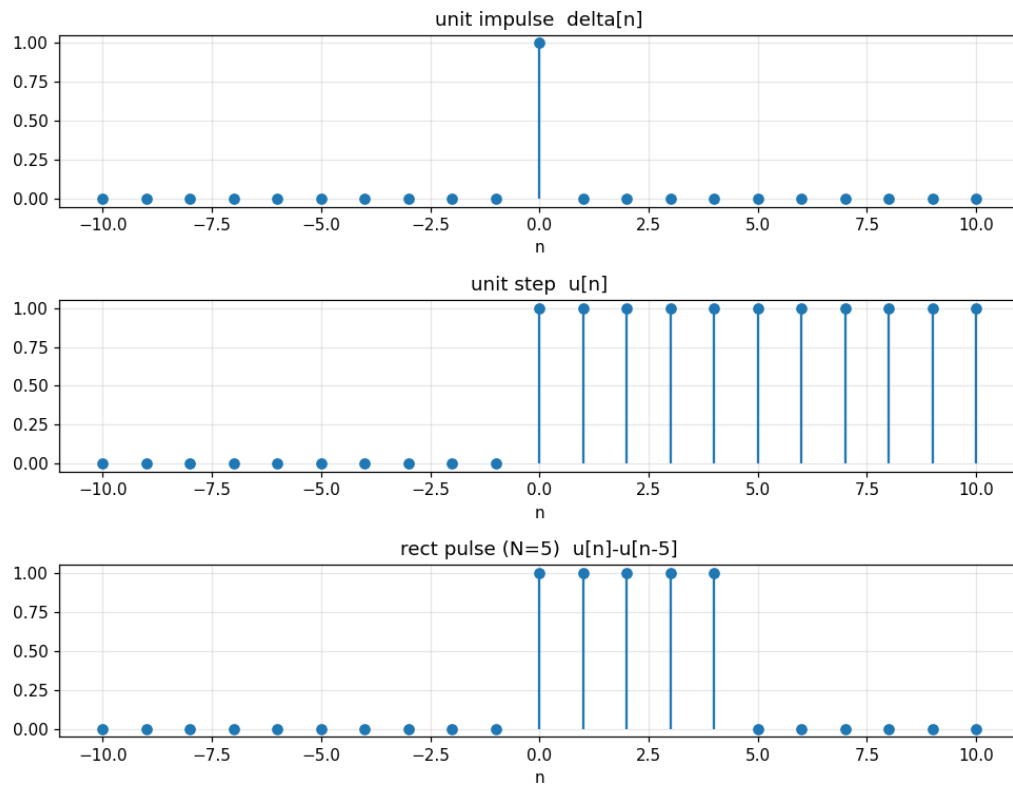
1  """P05 - Discrete-time building blocks. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6
7  # =====
8  # ANSWER IMPLEMENTATION
9  # =====
10 def unit_impulse(n):
11     """delta[n] = 1 at n==0, else 0."""
12     return np.where(n == 0, 1.0, 0.0)
13
14
15 def unit_step(n):
16     """u[n] = 1 for n >= 0, else 0."""
17     return np.where(n >= 0, 1.0, 0.0)
18
19
20 def rect_pulse_dt(n, N):
21     """Rectangular pulse: 1 for 0 <= n < N, else 0. Build as u[n] - u[n-N]."""
22     return unit_step(n) - unit_step(n - N)
23
24
25 def stem(ax, n, x, title):
26     ml, sl, bl = ax.stem(n, x)
27     bl.set_visible(False)
28     ax.set_title(title); ax.set_xlabel("n"); ax.grid(True, alpha=0.3)
29
30
31 def main():
32     n = np.arange(-10, 11)
33     fig, axs = plt.subplots(3, 1, figsize=(9, 7))
34     stem(axs[0], n, unit_impulse(n), "unit impulse delta[n]")
35     stem(axs[1], n, unit_step(n), "unit step u[n]")
36     stem(axs[2], n, rect_pulse_dt(n, 5), "rect pulse (N=5) u[n]-u[n-5]")
37     plt.show()
38
39     assert unit_impulse(n).sum() == 1.0
40     assert unit_step(np.array([-1, 0, 3])).tolist() == [0.0, 1.0, 1.0]
41     assert rect_pulse_dt(n, 5).sum() == 5.0 # ones at n=0..4
42     print("[P05] impulse sums to 1, step correct, rect has 5 ones -> passed.")
43

```

```
44  
45 if __name__ == "__main__":  
46     main()
```

Console output:

```
[P05] impulse sums to 1, step correct, rect has 5 ones -> passed.
```



Problem 06 — Time Shift by Interpolation

Problem 06

(Marks: 10 Time: 20 min)

Delay or advance a sampled signal, $y(t) = x(t - t_0)$, using `np.interp`. Because the shifted query times fall between stored samples, interpolation fills the gaps. A positive t_0 delays the signal (moves it to the right).

Tasks.

1. Implement `shift_signal(t, x, t0)` returning $x(t - t_0)$ on the same grid via `np.interp`, zero outside the support.
2. Plot $x(t)$, the delay $x(t - 2)$, and the advance $x(t + 2)$.

Marks Distribution: shift by interpolation 5; correct delay vs advance 2; plot 3.

Solution

`np.interp(t - t0, t, x, left=0, right=0)` reads the original samples at the shifted query points. Watch the sign: subtracting t_0 inside the argument moves the graph to the right (a delay).

```

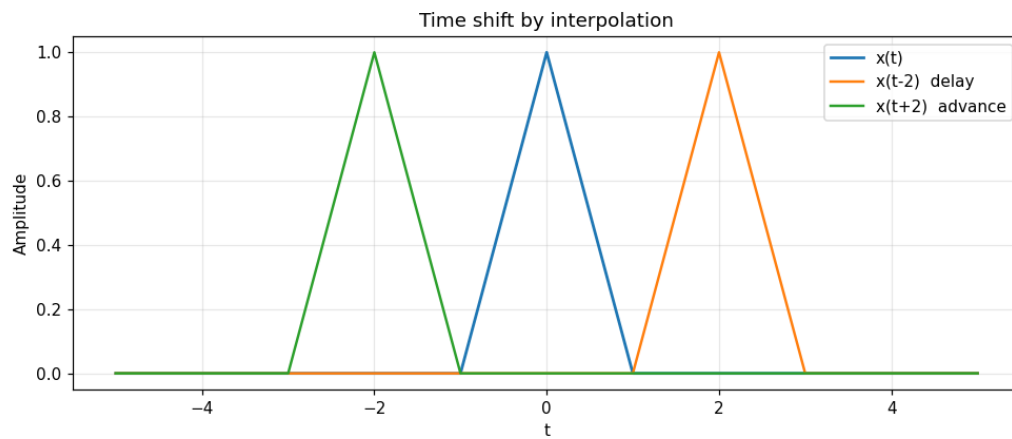
1  """P06 - Time shift of a sampled signal via interpolation. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6  T_MIN, T_MAX, N = -5.0, 5.0, 2001
7
8
9  def x_of_t(t):
10     """Given: a triangular bump centred at 0, width 2 (localised so a shift
11     is easy to see)."""
12     y = 1.0 - np.abs(t)
13     return np.where(np.abs(t) <= 1.0, y, 0.0)
14
15
16  # =====
17  # ANSWER IMPLEMENTATION
18  # =====
19  def shift_signal(t, x, t0):
20     """Return  $y(t) = x(t - t_0)$  sampled on the same grid  $t$ , using linear
21     interpolation. Values outside the original support are 0.
22     A positive  $t_0$  DELAYS the signal (moves it to the right)."""
23     return np.interp(t - t0, t, x, left=0.0, right=0.0)
24
25
26  def main():
27     t = np.linspace(T_MIN, T_MAX, N)
28     x = x_of_t(t)
29
30     fig, ax = plt.subplots(figsize=(9, 4))
31     ax.plot(t, x, label="x(t)", lw=1.8)
32     ax.plot(t, shift_signal(t, x, 2.0), label="x(t-2)  delay", lw=1.6)
33     ax.plot(t, shift_signal(t, x, -2.0), label="x(t+2)  advance", lw=1.6)
34     ax.set_title("Time shift by interpolation")
35     ax.set_xlabel("t"); ax.set_ylabel("Amplitude")
36     ax.grid(True, alpha=0.3); ax.legend()
37     plt.show()
38
39     # peak of x is at t=0; after delay by 2 the peak should be near t=2
40     y = shift_signal(t, x, 2.0)
41     assert abs(t[np.argmax(y)] - 2.0) < 0.05
42     ya = shift_signal(t, x, -2.0)
43     assert abs(t[np.argmax(ya)] + 2.0) < 0.05
44     print("[P06] peak moves to t=+2 for a delay and t=-2 for an advance")
45     " -> passed."

```

```
46  
47  
48 if __name__ == "__main__":  
49     main()
```

Console output:

```
[P06] peak moves to t=+2 for a delay and t=-2 for an advance -> passed.
```



Problem 07 — Downsampling a Discrete-Time Signal

Problem 07

(Marks: 10 Time: 15 min)

Time scaling in discrete time is done by keeping every M -th sample: $y[k] = x[Mk]$. This is pure array slicing with a step.

Tasks.

1. Implement `downsample(n, x, M)` returning the new index array and the kept values (use slicing `[:, :M]`).
2. Stem-plot the original signal and its $M = 2$ and $M = 3$ downsamplings.

Marks Distribution: downsample 5; index handling 2; stem plots 3.

Solution

Slicing with a step, `x[:, :M]`, keeps samples $0, M, 2M, \dots$. Remember to downsample the index array the same way so the stems land at the right positions.

```

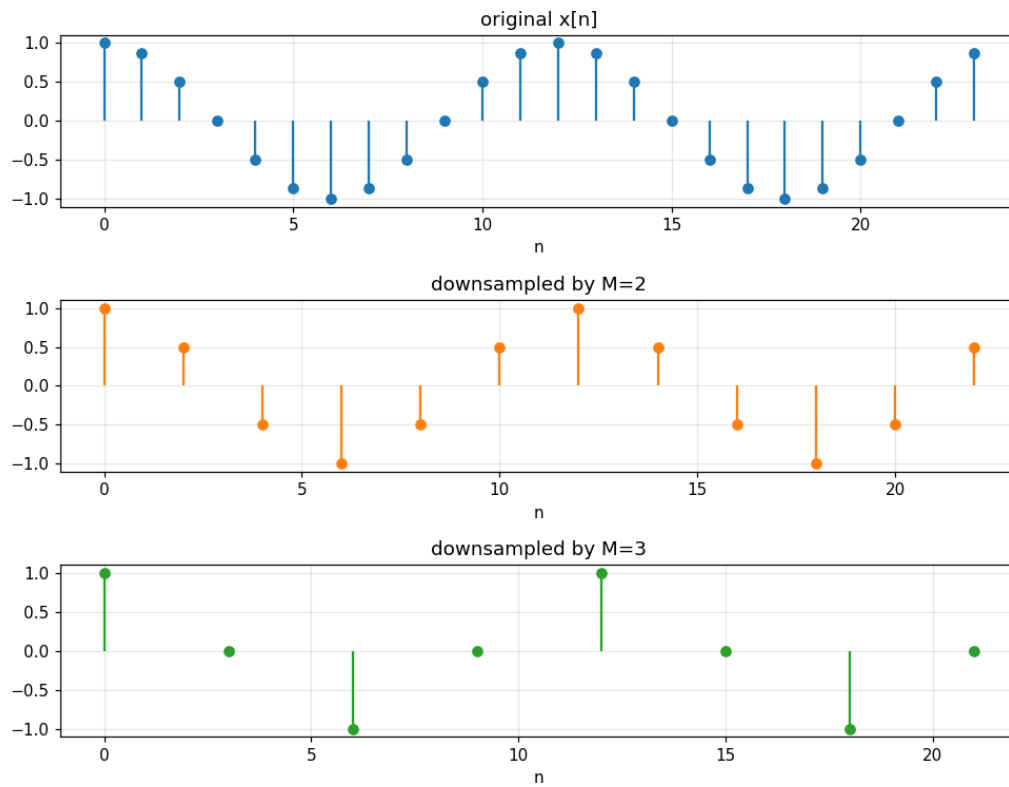
1  """P07 - Downsampling a discrete-time signal. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6
7  def x_of_n(n):
8      """Given: a slow discrete cosine."""
9      return np.cos(2 * np.pi * n / 12)
10
11
12  # =====
13  # ANSWER IMPLEMENTATION
14  # =====
15  def downsample(n, x, M):
16      """Keep every M-th sample:  $y[k] = x[Mk]$ . Return the new index array and
17      the downsampled values. (This is array slicing with a step.)"""
18      return n[:, :M], x[:, :M]
19
20
21  def stem(ax, n, x, title, color=None):
22      ml, sl, bl = ax.stem(n, x)
23      bl.set_visible(False)
24      if color:
25          ml.set_color(color); sl.set_color(color)
26      ax.set_title(title); ax.set_xlabel("n"); ax.grid(True, alpha=0.3)
27
28
29  def main():
30      n = np.arange(0, 24)
31      x = x_of_n(n)
32
33      n2, x2 = downsample(n, x, 2)
34      n3, x3 = downsample(n, x, 3)
35
36      fig, axs = plt.subplots(3, 1, figsize=(9, 7))
37      stem(axs[0], n, x, "original x[n]")
38      stem(axs[1], n2, x2, "downsampled by M=2", color="C1")
39      stem(axs[2], n3, x3, "downsampled by M=3", color="C2")
40      plt.show()
41
42      assert len(x2) == len(x[:, :2])
43      assert np.allclose(x2, x[:, :2])
44      assert np.allclose(x3, x[:, :3]) and np.array_equal(n3, n[:, :3])
45      print(f"[P07] M=2 keeps {len(x2)} of {len(x)} samples, "
46            f"    f"M=3 keeps {len(x3)} -> passed.")
47

```

```
48  
49 if __name__ == "__main__":  
50     main()
```

Console output:

```
[P07] M=2 keeps 12 of 24 samples, M=3 keeps 8 -> passed.
```



Problem 08 — Energy of a Discrete-Time Signal

Problem 08

(Marks: 10 Time: 20 min)

The energy of a discrete-time signal is $E = \sum_n |x[n]|^2$. Compare a finite rectangular pulse with a decaying exponential, and watch how the running (cumulative) energy settles.

Tasks.

1. Implement `energy_dt(x) =  $\sum |x[n]|^2$` .
2. Implement `cumulative_energy(x)` (a running total via `np.cumsum`).
3. Print both energies against their exact values and plot the cumulative energy.

Marks Distribution: energy 4; cumulative 3; print and plot 3.

Solution

A height-2, 5-sample pulse has energy $5 \cdot 2^2 = 20$. The geometric signal $(0.8)^n u[n]$ has energy $\sum (0.8)^{2n} = 1/(1-0.64) \approx 2.78$: its cumulative energy flattens out, the signature of a finite-energy signal.

```

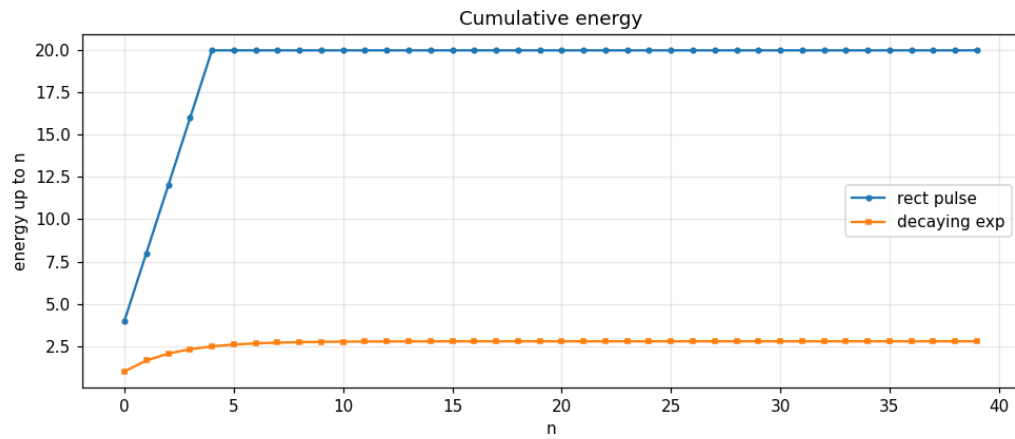
1  """P08 - Energy of a discrete-time signal. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6
7  def pulse(n):
8      """Given: a 5-sample rectangular pulse of height 2 (finite energy)."""
9      return np.where((n >= 0) & (n < 5), 2.0, 0.0)
10
11
12  def decay(n):
13      """Given: a decaying exponential (0.8)^n u[n], truncated to n>=0."""
14      return np.where(n >= 0, 0.8 ** n, 0.0)
15
16
17  # =====
18  # ANSWER IMPLEMENTATION
19  # =====
20  def energy_dt(x):
21      """Total energy of a discrete-time signal: sum of |x[n]|^2."""
22      return float(np.sum(np.abs(x) ** 2))
23
24
25  def cumulative_energy(x):
26      """Running energy: element k is the energy of x[0..k]."""
27      return np.cumsum(np.abs(x) ** 2)
28
29
30  def main():
31      n = np.arange(0, 40)
32      xp, xd = pulse(n), decay(n)
33
34      Ep, Ed = energy_dt(xp), energy_dt(xd)
35      print(f"[P08] pulse energy = {Ep:.4f} (exact 5*2^2 = 20)")
36      print(f"[P08] decay energy = {Ed:.4f} (exact 1/(1-0.64) = {1/(1-0.64):.4f})")
37
38      fig, ax = plt.subplots(figsize=(9, 4))
39      ax.plot(n, cumulative_energy(xp), "o-", label="rect pulse", ms=3)
40      ax.plot(n, cumulative_energy(xd), "s-", label="decaying exp", ms=3)
41      ax.set_title("Cumulative energy"); ax.set_xlabel("n")
42      ax.set_ylabel("energy up to n"); ax.grid(True, alpha=0.3); ax.legend()
43      plt.show()
44
45  assert np.isclose(Ep, 20.0)

```

```
46     assert np.isclose(Ed, 1 / (1 - 0.64), atol=1e-3)
47     print("[P08] energies match the exact values -> passed.")
48
49
50 if __name__ == "__main__":
51     main()
```

Console output:

```
[P08] pulse energy    = 20.0000  (exact 5*2^2 = 20)
[P08] decay energy    = 2.7778  (exact 1/(1-0.64) = 2.7778)
[P08] energies match the exact values -> passed.
```



Problem 09 — Average Power of a Periodic Signal

Problem 09

(Marks: 10 Time: 20 min)

For a periodic signal the average power over one fundamental period equals the average over any whole number of periods: $P = \frac{1}{N_0} \sum_{n=0}^{N_0-1} |x[n]|^2$. Verify this for a cosine of period $N_0 = 8$.

Tasks.

1. Implement `average_power_one_period`: the mean of $|x|^2$ over one period.
2. Implement `average_power`: the mean of $|x|^2$ over the whole array.
3. Show the one-period and five-period powers agree and equal the exact cosine power $\frac{1}{2}$; stem-plot with one period shaded.

Marks Distribution: one-period power 4; full power 2; match to 0.5 2; plot 2.

Solution

A cosine spends half its time-average on $\cos^2$, whose mean is $\frac{1}{2}$. Because the signal repeats, one period already contains all the information – averaging over more periods changes nothing.

```

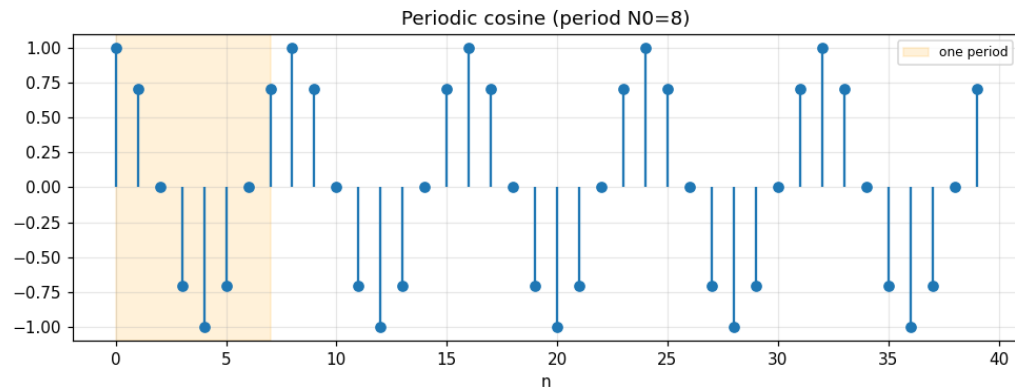
1  """P09 - Average power of a periodic discrete-time signal. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6  N0 = 8    # fundamental period
7
8
9  def x_of_n(n):
10     """Given: a periodic cosine with period N0 = 8."""
11     return np.cos(2 * np.pi * n / N0)
12
13
14  # =====
15  # ANSWER IMPLEMENTATION
16  # =====
17  def average_power_one_period(x_period):
18     """Average power over exactly one period: (1/N0) * sum |x|^2."""
19     return float(np.mean(np.abs(x_period) ** 2))
20
21
22  def average_power(x):
23     """Average power over the whole array: mean of |x|^2."""
24     return float(np.mean(np.abs(x) ** 2))
25
26
27  def main():
28     n_one = np.arange(0, N0)          # one period
29     n_many = np.arange(0, 5 * N0)     # five periods
30
31     p1 = average_power_one_period(x_of_n(n_one))
32     pK = average_power(x_of_n(n_many))
33     print(f"[P09] power over 1 period = {p1:.6f}")
34     print(f"[P09] power over 5 periods = {pK:.6f}")
35     print(f"[P09] exact value for a cosine = 0.5")
36
37     fig, ax = plt.subplots(figsize=(9, 3.6))
38     m1, s1, bl = ax.stem(n_many, x_of_n(n_many)); bl.set_visible(False)
39     ax.axvspan(0, N0 - 1, color="orange", alpha=0.15, label="one period")
40     ax.set_title("Periodic cosine (period N0=8)")
41     ax.set_xlabel("n"); ax.grid(True, alpha=0.3); ax.legend(fontsize=8)
42     plt.show()
43
44     assert np.isclose(p1, pK)          # one period is enough
45     assert np.isclose(p1, 0.5)        # matches the exact cosine power

```

```
46     print("[P09] one-period power equals full power equals 0.5  -> passed.")
47
48
49 if __name__ == "__main__":
50     main()
```

Console output:

```
[P09] power over 1 period = 0.500000
[P09] power over 5 periods = 0.500000
[P09] exact value for a cosine = 0.5
[P09] one-period power equals full power equals 0.5  -> passed.
```



Problem 10 — Sampling a Continuous-Time Sinusoid

Problem 10

(Marks: 10 Time: 20 min)

Turn a continuous cosine into a discrete set of samples taken at rate f_s . Overlay the dense continuous curve with the samples, and note the Nyquist rule: to represent a frequency f you need $f_s > 2f$.

Tasks.

1. Implement `continuous_signal(t, f) = cos(2 $\pi$ ft)` on a dense grid.
2. Implement `sample_times(fs, duration)` giving instants $0, \frac{1}{f_s}, \frac{2}{f_s}, \dots$
3. Implement `sample_signal(f, fs, duration)` and plot the continuous curve with stems at $f_s = 20$ and $f_s = 5$ Hz.

Marks Distribution: continuous 2; sample times 3; samples 2; plot 3.

Solution

Sampling at f_s places samples $1/f_s$ seconds apart. For a 2 Hz cosine, $f_s = 20$ Hz captures the shape well, while $f_s = 5$ Hz is close to the Nyquist limit $2f = 4$ Hz and looks coarse.

```

1  """P10 - Sampling a continuous-time sinusoid. Reference solution."""
2  import numpy as np
3  import matplotlib
4  import matplotlib.pyplot as plt
5
6
7  # =====
8  # ANSWER IMPLEMENTATION
9  # =====
10 def continuous_signal(t, f):
11     """A 'continuous' cosine cos(2*pi*f*t) evaluated on a dense grid t."""
12     return np.cos(2 * np.pi * f * t)
13
14
15 def sample_times(fs, duration):
16     """Sample instants 0, 1/fs, 2/fs, ... up to (but not including) duration."""
17     return np.arange(0, duration, 1.0 / fs)
18
19
20 def sample_signal(f, fs, duration):
21     """Return (sample_times, samples) of cos(2*pi*f*t) taken at rate fs."""
22     ts = sample_times(fs, duration)
23     return ts, continuous_signal(ts, f)
24
25
26 def main():
27     f, duration = 2.0, 1.0
28     t_dense = np.linspace(0, duration, 1000)
29
30     fig, ax = plt.subplots(figsize=(9, 4))
31     ax.plot(t_dense, continuous_signal(t_dense, f),
32            "0.6", label="continuous cos(2*pi*2*t)")
33     for fs, mark in [(20.0, "C0"), (5.0, "C3")]:
34         ts, xs = sample_signal(f, fs, duration)
35         ml, sl, bl = ax.stem(ts, xs, linefmt=mark, markerfmt=mark + "o")
36         bl.set_visible(False)
37         ml.set_label(f"samples at fs={fs:.0f} Hz")
38     ax.set_title("Sampling a continuous-time sinusoid (f=2 Hz)")
39     ax.set_xlabel("t"); ax.set_ylabel("Amplitude")
40     ax.grid(True, alpha=0.3); ax.legend(fontsize=8)
41     plt.show()
42
43     ts, xs = sample_signal(f, 20.0, duration)
44     assert len(ts) == 20 and np.isclose(ts[1] - ts[0], 1 / 20)

```

```

45     assert np.isclose(xs[0], 1.0)                # cos(0) = 1
46     print(f"[P10] fs=20 Hz gives {len(ts)} samples spaced {1/20:.3f}s; "
47           f"Nyquist needs fs > 2f = 4 Hz -> passed.")
48
49
50 if __name__ == "__main__":
51     main()

```

Console output:

```
[P10] fs=20 Hz gives 20 samples spaced 0.050s; Nyquist needs fs > 2f = 4 Hz -> passed.
```

